

CajuHub · Formação Dev Junior

EventoAqui

Plataforma de Venda de Ingressos

Banco de Dados com PostgreSQL

Squad · Kivia · Kauan · Kauã · Isac · Gabriel · Vitor

Projeto Final · 2026

EA

Por que o banco existe?



Problema

Cada organizador gerenciava seus eventos em planilhas próprias — sem padrão, sem visibilidade central.



Solução

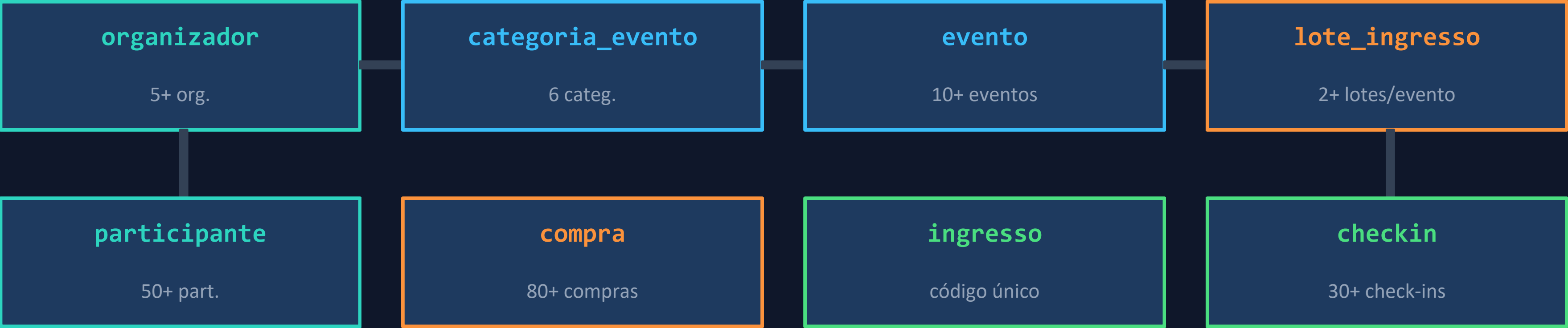
Um banco central que conecta organizadores, participantes, ingressos e check-ins em uma única plataforma.



Resultado

Controle de capacidade, rastreamento de cancelamentos e relatórios operacionais em tempo real.

8 Tabelas · Diagrama ER



Cardinalidades

1 : N

organizador → evento

— Um org. publica vários eventos

1 : N

evento → lote_ingresso

— Um evento tem múltiplos lotes

1 : N

participante → compra

— Um participante faz várias compras

1 : N

compra → ingresso

— Uma compra tem vários ingressos

1 : 1

ingresso → checkin

— Cada ingresso tem no máx. 1 check-in

Decisões de Modelagem



```
CREATE TABLE ingresso (  
  id SERIAL PRIMARY KEY,  
  codigo VARCHAR(20)  
    UNIQUE NOT NULL,  
  status VARCHAR(20)  
    CHECK (status IN (  
      'valido',  
      'utilizado',  
      'cancelado'))),  
  compra_id INT REFERENCES  
    compra(id)  
    ON DELETE RESTRICT,  
  lote_id INT REFERENCES  
    lote_ingresso(id)  
    ON DELETE RESTRICT  
);
```

Por que lote e ingresso são entidades separadas?

Lote define tipo e preço. Ingresso é a unidade física vendida com código único.

Como o código único é garantido?

codigo VARCHAR(20) UNIQUE NOT NULL — o banco rejeita duplicatas no nível do schema.

O que impede deletar um evento com compras?

ON DELETE RESTRICT em todas as FKs — nenhum pai pode ser removido se tiver filhos.

Por que check-in é uma tabela separada?

Registra timestamp e responsável. Um campo de status perderia o histórico e o quem validou.

Volume e Coerência dos Dados

5+

Organizadores

(1 suspenso)

5

Categorias

show, festival, workshop...

10+

Eventos

status variados, 2 meses

50+

Participantes

CPF e e-mail únicos

80+

Compras

status variados

30+

Check-ins

eventos encerrados

10+

Cancelamentos

ou estornos rastreados

∞

Códigos únicos

sem repetição possível

12 Perguntas de Negócio · Q1–Q6



-- Q2: Ocupação por lote

SELECT

```
evento.titulo,  
lote.nome,  
lote.capacidade_maxima,  
COUNT(ingresso.id) vendidos,  
ROUND(  
    COUNT(ingresso.id) * 100.0  
    / lote.capacidade_maxima, 2  
) AS "Ocupação (%)"
```

FROM lote_ingresso lote

INNER JOIN evento ON ...

LEFT JOIN ingresso ON ...

```
AND ingresso.status IN (  
    'valido','utilizado')
```

GROUP BY evento.titulo,

lote.nome, lote.capacidade_maxima

ORDER BY "Ocupação (%)" DESC;

Q1

Eventos publicados com data futura

Q2

Ocupação % de cada lote por evento

Q3

Participantes em mais de 1 evento

Q4

Receita total por evento (aprovadas)

Q5

Eventos com pelo menos 1 lote esgotado

Q6

% de presença em eventos encerrados

12 Perguntas de Negócio · Q7–Q12



-- Q12: Receita por vaga

SELECT

```
e.titulo,  
SUM(l.capacidade_maxima)  
  AS "Capacidade Total",  
ROUND(  
  COALESCE(SUM(c.valor_total),0)  
  / SUM(l.capacidade_maxima), 2  
) AS "Receita por Vaga"
```

FROM evento e

INNER JOIN lote_ingresso l

ON e.id = l.evento_id

LEFT JOIN ingresso i ON l.id = i.lote_id
AND i.status IN ('valido','utilizado')

LEFT JOIN compra c ON i.compra_id = c.id
AND c.status = 'aprovado'

GROUP BY e.id, e.titulo

HAVING SUM(l.capacidade_maxima) > 0

ORDER BY "Receita por Vaga" DESC LIMIT 5;

Q7

Participantes sem check-in em eventos encerrados

Q8

Organizador com maior receita acumulada

Q9

Eventos com mais cancelamentos que check-ins

Q10

Categoria mais popular (últimos 60 dias)

Q11

Compras pendentes há mais de 2 dias

Q12

Top 5 eventos por receita por vaga

O que aplicamos nas queries

INNER JOIN

Q1, Q4, Q8

Garante que ambos os lados existam (ex: evento + organizador)

LEFT JOIN

Q2, Q6, Q7

Traz todos da esquerda mesmo sem match (ex: lotes sem ingressos vendidos)

COUNT(DISTINCT)

Q3, Q6

Conta valores únicos — evita contar o mesmo evento múltiplas vezes

COALESCE(..., 0)

Q4, Q12

Substitui NULL por zero quando não há compras

HAVING

Q3, Q9, Q12

Filtra grupos após o GROUP BY — usado com funções de agregação

NOW() - INTERVAL

Q1, Q10, Q11

Filtra por janela temporal (data futura, últimos 60 dias, +2 dias)

ROUND(..., 2)

Q2, Q6, Q12

Arredonda percentuais e receita por vaga com 2 casas decimais

ORDER BY + LIMIT

Q8, Q10, Q12

Ranking — retorna apenas o top N sem varredura desnecessária

Relatórios Recorrentes Encapsulados

VIEW 1 • painel_eventos_ativos

```
SELECT e.titulo, o.nome, e.data_inicio,  
       e.cidade, cat.nome,  
       COUNT(i.id) AS ingressos_vendidos,  
       SUM(l.capacidade_maxima) - COUNT(i.id)  
         AS capacidade_restante  
FROM evento e ...  
WHERE e.status = 'publicado'  
GROUP BY e.id, ...  
ORDER BY e.data_inicio ASC;
```

Uso: atualizado automaticamente a cada consulta — sem reescrever a query.

Columns: ID Evento · Evento · Organizador · Data Início · Cidade · Categoria · Ingressos Vendidos · Capacidade Restante

⚡ Novo ingresso vendido? A capacidade restante já reflete na próxima SELECT.

VIEW 2 • receita_por_organizador

```
SELECT o.nome,  
       COUNT(DISTINCT e.id) AS total_eventos,  
       COUNT(DISTINCT c.id) AS compras_aprov,  
       COALESCE(SUM(c.valor_total), 0) AS receita  
FROM organizador o ...  
HAVING COUNT(DISTINCT c.id) > 0  
ORDER BY receita DESC;
```

HAVING garante: apenas organizadores com ≥ 1 evento E ≥ 1 compra aprovada. Eventos cancelados não entram — apenas os que geraram receita real.

Atomicidade no Cancelamento



```
-- 1) Verificar ANTES
SELECT c.status, i.status
FROM compra c JOIN ingresso i ...
WHERE c.id = 4;

-- 2) Abrir transaction
BEGIN;

-- Op. 1: cancelar compra
UPDATE compra SET status = 'cancelado'
WHERE id = 4;

-- Op. 2: cancelar ingressos
UPDATE ingresso SET status = 'cancelado'
WHERE compra_id = 4;

-- 3) Forçar ROLLBACK (demo)
ROLLBACK; -- ← tudo desfeito

-- 4) Verificar DEPOIS
-- status volta a 'aprovado' / 'valido'
```

1

Atualizar compra

SET status = 'cancelado' na tabela compra WHERE id = X

2

Cancelar ingressos

Todos os ingressos WHERE compra_id = X recebem status 'cancelado'

3

Vagas liberadas

Como os ingressos saem do filtro 'valido/utilizado', a capacidade retorna automaticamente

⚠ ROLLBACK forçado

Se qualquer operação falhar → NENHUMA persiste

Checklist de Conformidade com o Briefing

✓	Schema	schema.sql roda do zero sem erros — todas as constraints e FKs
✓	Seed	5 org., 5 cat., 10+ eventos, 50+ participantes, 80+ compras, 30+ check-ins
✓	Queries	12 perguntas de negócio respondidas com comentários identificadores
✓	Views	painel_eventos_ativos e receita_por_organizador criadas e testadas
✓	Transaction	BEGIN/ROLLBACK demonstrado com compra 4 (2 ingressos vinculados)
✓	GitHub	33 commits com mensagens descritivas, histórico de múltiplos membros
✓	UNIQUE	codigo UNIQUE NOT NULL no ingresso — sem duplicatas possíveis
✓	ON DELETE	RESTRICT em todas as FKs — delete em cascata bloqueado pelo banco
✓	README	Diagrama ER textual e decisões de modelagem no README.md

Obrigado!

EventoAqui · Banco de Dados PostgreSQL

- 8 tabelas · 120 linhas de schema
- 12 queries de negócio respondidas
- 2 views operacionais
- Transaction com ROLLBACK demonstrado
- 33 commits no GitHub

Squad CajuHub · 2026

github.com/Star-Kivia/EventoAqui

The EA logo is displayed in a light blue color. It consists of the letters 'E' and 'A' in a bold, sans-serif font. The 'E' has three horizontal bars, and the 'A' is a simple triangle shape. The logo is positioned on the right side of the slide, within a dark blue circular background element.